

Transaction Level Modeling (TLM) using SystemC

PROGRAM:

```
#include <systemc>

#include <tlm>

#include <tlm_utils/simple_initiator_socket.h>
#include <tlm_utils/simple_target_socket.h>

using namespace sc_core;
using namespace tlm;
using namespace std;

// Target
struct Target : sc_module {
    tlm_utils::simple_target_socket<Target> socket;

    SC_CTOR(Target) : socket("socket") {
        socket.register_b_transport(this, &Target::b_transport);
    }

    void b_transport(tlm_generic_payload& trans, sc_time& delay) {
        int* data = (int*) trans.get_data_ptr();
        cout << "Target received: " << *data << endl;
        delay += sc_time(10, SC_NS);
    }
};

// Initiator
struct Initiator : sc_module {
    tlm_utils::simple_initiator_socket<Initiator> socket;

    SC_CTOR(Initiator) : socket("socket") {
        SC_THREAD(thread_process);
    }
};
```

```
}
```

```
void thread_process() {
```

```
    tlm_generic_payload trans;
```

```
    sc_time delay = SC_ZERO_TIME;
```

```
    int data = 555;
```

```
    trans.set_data_ptr(reinterpret_cast<unsigned  
char*>(&data)); trans.set_data_length(sizeof(data));
```

```
    cout << "Initiator sending: " << data <<  
endl; socket->b_transport(trans, delay);
```

```
    wait(delay);
```

```
}
```

```
};
```

```
int sc_main(int argc, char* argv[]) {
```

```
    Initiator initiator("initiator");
```

```
    Target target("target");
```

```
    initiator.socket.bind(target.socket);
```

```
    sc_start();
```

```
    return 0;
```

```
}
```

OUTPUT:

The screenshot displays the EDA Playground web interface. The browser address bar shows the URL <https://www.edaplayground.com/x/5Vg4>. The page header includes the EDA Playground logo and navigation tabs: New, Run, Copy, Resources, Community, Help, Playgrounds, and Profile.

On the left sidebar, there is a section "Brought to you by DOULOS" with a disclaimer. Below it are links for "Languages & Libraries", "Tools & Simulators", and "Examples". The "Examples" list includes: using EDA Playground, VHDL, Verilog/SystemVerilog, UVM, EasierUVM, SVAUnit, SVUnit, VUnit (Verilog/SV), VUnit (VHDL), TL-Verilog, e + Verilog, Python + Verilog, Python + VHDL, Python Only, and C++/SystemC.

The main area contains two code editors. The left editor, titled "testbench.cpp", contains the following C++ code:

```
3 #include <tlm_utils/simple_initiator_socket.h>
4 #include <tlm_utils/simple_target_socket.h>
5 using namespace sc_core;
6 using namespace tlm;
7 using namespace std;
8 // Target
9 struct Target : sc_module {
10     tlm_utils::simple_target_socket<Target> socket;
11     SC_CTOR(Target) : socket("socket") {
12         socket.register_b_transport(this, &target::b_transport);
13     }
14     void b_transport(tlm_generic_payload& trans, sc_time&
15         delay) {
16         int* data = (int*) trans.get_data_ptr();
17         cout << "Target received: " << *data << endl;
18         delay += sc_time(10, SC_NS);
19     }
20 };
21 // Initiator
22 struct Initiator : sc_module {
```

The right editor, titled "design.cpp", contains the following C++ code:

```
1 // Code your design here.
2 // Uncomment the next line for SystemC modules.
3 // #include <systemc>
```

Below the code editors is a terminal window with a "Log" button and a "Share" link. The terminal output shows the compilation and execution process:

```
[2026-08-25 03:44:30 UTC] g++ -o sim -lsystemc *.cpp && echo "Compile done. Starting run..." && ./sim
Compile done. Starting run...

SystemC 2.3.3-Accellera --- May 23 2020 14:33:56
Copyright (c) 1996-2018 by all Contributors,
ALL RIGHTS RESERVED

Initiator sending: 555
Target received: 555
Done
```

The bottom of the image shows a Windows taskbar with various application icons, a search bar, and system status icons including network, volume, and battery, along with the date and time (09:14, 25-08-2026).